

Авторские права
© Postgres Professional, 2017–2024
Авторы: Егор Рогов, Павел Лузанов, Илья Баштанов, Игорь Гнатюк
Фото: Олег Бартунов (монастырь Пху и пик Бхрикути, Непал)

Использование материалов курса
Некоммерческое использование материалов курса (презентации, демонстрации) разрешается без ограничений. Коммерческое использование возможно только с письменного разрешения компании Postgres Professional. Запрещается внесение изменений в материалы курса.

Обратная связь
Отзывы, замечания и предложения направляйте по адресу:
edu@postgrespro.ru

Отказ от ответственности
Компания Postgres Professional не несет никакой ответственности за любые повреждения и убытки, включая потерю дохода, нанесенные прямым или непрямым, специальным или случайным использованием материалов курса. Компания Postgres Professional не предоставляет каких-либо гарантий на материалы курса. Материалы курса предоставляются на основе принципа «как есть» и компания Postgres Professional не обязана предоставлять сопровождение, поддержку, обновления, расширения и изменения.

Темы

Триггеры и триггерные функции

В какой момент срабатывают триггеры

Контекст выполнения триггерной функции

Возвращаемое значение

Для чего можно применять триггеры и для чего не нужно

Событийные триггеры

Триггеры и функции

Триггер

объект базы данных — список обрабатываемых событий

при возникновении события вызывается триггерная функция и ей передается контекст

Триггерная функция

объект базы данных — код обработки события

выполняется в той же транзакции, что и основная операция

соглашение: функция не принимает параметры, возвращает значение псевдотипа trigger (фактически record)

может использоваться в нескольких триггерах

Механизм триггеров позволяет выполнять определенные действия «в ответ» на определенные события. Триггер состоит из двух частей: собственно триггера (который определяет события) и триггерной функции (которая определяет действия). И триггер, и функция являются самостоятельными объектами БД.

Когда возникает событие, на которое «подписан» триггер, вызывается триггерная функция. Ей передается контекст вызова, чтобы можно было определить, какой именно триггер и в каких условиях вызвал функцию.

Триггерная функция — это обычная функция, которая написана с учетом некоторых соглашений:

- она пишется на любом языке, кроме чистого SQL;
- она не имеет параметров;
- она возвращает значение типа trigger (на самом деле это псевдотип, по факту возвращается запись, соответствующая строке таблицы).

Триггерная функция выполняется в той же транзакции, что и основная операция. Таким образом, если триггерная функция завершится с ошибкой, вся транзакция будет прервана.

https://postgrespro.ru/docs/postgresql/16/trigger-definition

События

INSERT, UPDATE, DELETE

таблицы	before/after	statement
	before/after	row
представления	before/after	statement
	instead of	row

TRUNCATE

таблицы	before/after	statement

Условие WHEN

устанавливает дополнительный фильтр

Триггеры могут срабатывать на вставку (INSERT), обновление (UPDATE) или удаление (DELETE) строк в таблице или представлении, а также на опустошение (TRUNCATE) таблиц.

Триггер может срабатывать до выполнения действия (BEFORE), после него (AFTER), или вместо него (INSTEAD OF).

Триггер может срабатывать один раз для всей операции (FOR EACH STATEMENT), или каждый раз для каждой затронутой строки (FOR EACH ROW).

Не для любой комбинации этих условий можно создать триггер. Например, instead-of-триггеры можно определить только для представлений на уровне строк, а truncate-триггер можно определить только для таблиц и только на уровне оператора. Допустимые варианты перечислены на слайде.

Кроме того, можно сузить область действия триггера, указав дополнительное условие WHEN: если условие не выполняется — триггер не срабатывает.

https://postgrespro.ru/docs/postgresql/16/sql-createtrigger

Before statement

Срабатывает

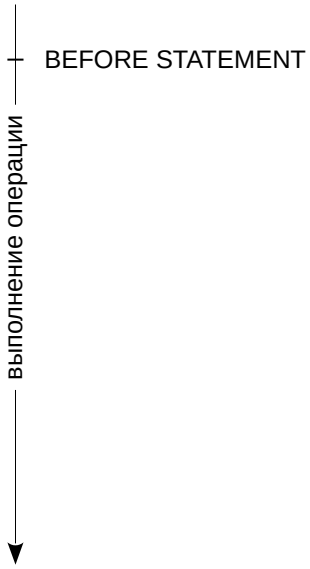
до операции

Возвращаемое значение

игнорируется

Контекст

TG-переменные



Рассмотрим подробнее разные типы триггеров.

Триггер BEFORE STATEMENT срабатывает один раз для операции независимо от того, сколько строк будет затронуто (возможно, что и ни одной). Это происходит до того, как операция начала выполняться.

Возвращаемое значение триггерной функции игнорируется. Если в триггере возникает ошибка, операция отменяется.

Поскольку триггерная функция не имеет параметров, контекст вызова в PL/pgSQL передается ей с помощью предопределенных TG-переменных, таких, как:

- TG_WHEN = «BEFORE»,
- TG_LEVEL = «STATEMENT»,
- TG_OP = «INSERT»/«UPDATE»/«DELETE»/«TRUNCATE»

и др. Триггерной функции можно также передать пользовательский контекст (аналог отсутствующих параметров) через переменную TG_ARGV, хотя зачастую лучше вместо одной «обобщенной» функции создать несколько «частных».

https://postgrespro.ru/docs/postgresql/16/plpgsql-trigger

Before row

Срабатывает

перед действием со строкой
в процессе выполнения операции

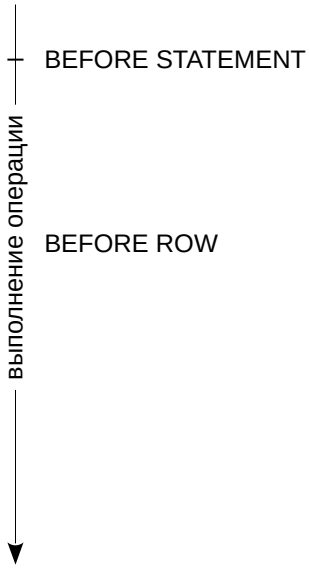
Возвращаемое значение

строка (возможно, измененная)
null отменяет действие

Контекст

OLD update, delete
NEW insert, update

TG-переменные



Триггеры BEFORE ROW срабатывают каждый раз перед тем, как операция затронет строку. Это происходит непосредственно в процессе выполнения операции.

В качестве контекста триггерная функция получает переменные:

- OLD — старая строка (не определено для операции вставки),
- NEW — измененная строка (не определено для удаления),
- TG_WHEN = «BEFORE»,
- TG_LEVEL = «ROW»,
- TG_OP = «INSERT»/«UPDATE»/«DELETE»

и др.

Возврат неопределенного значения NULL воспринимается как отмена действия над данной строкой. Сама операция продолжит выполнение, но текущая строка не будет обработана и другие триггеры для этой строки не сработают.

Чтобы не вмешиваться в работу операции, триггер должен вернуть строку в том виде, в котором ее собирается изменить операция: NEW для вставки и обновления, любое значение (но не NULL) для удаления (обычно используют OLD).

Но триггерная функция может и изменить значение NEW, чтобы повлиять на результат операции — часто именно для этого такой триггер и создают.

Instead of row

Срабатывает

вместо действия со строкой
для представлений

Возвращаемое значение

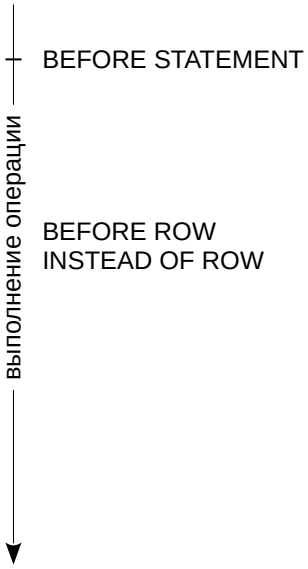
строка (возможно, измененная) —
будет видна в RETURNING

null отменяет действие

Контекст

OLD	update, delete
NEW	insert, update

TG-переменные



Триггеры INSTEAD OF очень похожи на триггеры BEFORE, но определяются только для представлений и срабатывают не до, а вместо операции.

В задачу таких триггеров обычно входит выполнение необходимых операций над базовыми таблицами представления. Также триггер может вернуть измененное значение NEW — именно оно будет видно при выполнении операции с указанием фразы RETURNING.

After row

Postgres PROFESSIONAL

Срабатывает

после выполнения операции

очередь из прошедших условие WHEN

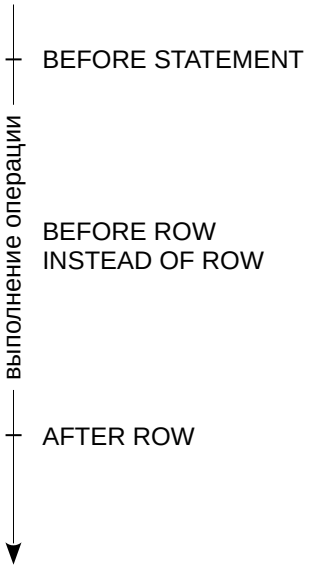
Возвращаемое значение

игнорируется

Контекст

OLD, OLD TABLE		update, delete
NEW, NEW TABLE	insert,	update

TG-переменные



Триггеры AFTER ROW, как и BEFORE ROW, срабатывают для каждой затрагиваемой строки, но не сразу после действия над строкой, а после того, как выполнена вся операция - чтобы при обращении из этих триггеров к изменяющейся таблице результат не зависел от порядка обработки строк. Для этого события сначала помещаются в очередь и обрабатываются после окончания операции. Чем меньше событий попадет в очередь, тем меньше будет накладных расходов — поэтому именно в этом случае очень полезно использовать условие WHEN, чтобы отсечь заведомо ненужные строки.

Возвращаемое значение триггеров AFTER ROW игнорируется (поскольку операция уже выполнена).

Контекст триггерной функции составляют:

- OLD — старая строка (не определено для операции вставки),
- NEW — новое значение строки (не определено для удаления).

Кроме этих переменных, начиная с версии 10 триггерная функция может получить доступ к специальным *переходным таблицам* (transition tables). Таблица, указанная при создании триггера как OLD TABLE, содержит старые значения строк, обработанных триггером, а таблица NEW TABLE — новые значения тех же строк.

Доступны и обычные TG-переменные, включая:

- TG_WHEN = «AFTER»,
- TG_LEVEL = «ROW»,
- TG_OP = «INSERT»/«UPDATE»/«DELETE».

After statement

Срабатывает

после операции
(даже если не затронута ни одна строка)

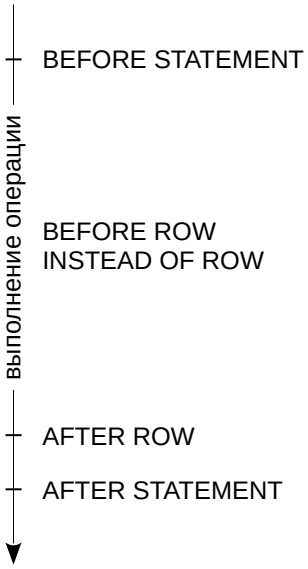
Возвращаемое значение

игнорируется

Контекст

OLD TABLE update, delete
NEW TABLE insert, update

TG-переменные



Триггер AFTER STATEMENT срабатывает один раз после окончания операции и после всех триггеров AFTER ROW (независимо от того, сколько строк было затронуто).

Возвращаемое значение триггерной функции игнорируется.

Контекст вызова передается с помощью переходных таблиц. Обращаясь к ним, триггерная функция может проанализировать все затронутые строки. Обычно переходные таблицы используются именно с триггерами AFTER STATEMENT, а не AFTER ROW.

Также определены обычные TG-переменные:

- TG_WHEN = «AFTER»,
- TG_LEVEL = «STATEMENT»,
- TG_OP = «INSERT»/«UPDATE»/«DELETE»/«TRUNCATE»

и др.

Порядок вызова триггеров

Создадим «универсальную» триггерную функцию, которая описывает контекст, в котором она вызвана. Контекст передается в различных TG-переменных.

Затем создадим триггеры на различные события, и будем смотреть, какие триггеры и в каком порядке вызываются при выполнении операций.

```
=> CREATE OR REPLACE FUNCTION describe() RETURNS trigger
AS $$
DECLARE
    rec record;
    str text := '';
BEGIN
    IF TG_LEVEL = 'ROW' THEN
        CASE TG_OP
            WHEN 'DELETE' THEN rec := OLD; str := OLD::text;
            WHEN 'UPDATE' THEN rec := NEW; str := OLD || ' -> ' || NEW;
            WHEN 'INSERT' THEN rec := NEW; str := NEW::text;
        END CASE;
    END IF;
    RAISE NOTICE '% % % %: %',
        TG_TABLE_NAME, TG_WHEN, TG_OP, TG_LEVEL, str;
    RETURN rec;
END
$$ LANGUAGE plpgsql;
```

```
CREATE FUNCTION
```

Таблица:

```
=> CREATE TABLE t(
    id integer PRIMARY KEY,
    s text
);
```

```
CREATE TABLE
```

Триггеры на уровне оператора.

```
=> CREATE TRIGGER t_before_stmt
BEFORE INSERT OR UPDATE OR DELETE -- события
ON t                              -- таблица
FOR EACH STATEMENT                -- уровень
EXECUTE FUNCTION describe();      -- триггерная функция
```

```
CREATE TRIGGER
```

```
=> CREATE TRIGGER t_after_stmt
AFTER INSERT OR UPDATE OR DELETE ON t
FOR EACH STATEMENT EXECUTE FUNCTION describe();
```

```
CREATE TRIGGER
```

И на уровне строк:

```
=> CREATE TRIGGER t_before_row
BEFORE INSERT OR UPDATE OR DELETE ON t
FOR EACH ROW EXECUTE FUNCTION describe();
```

```
CREATE TRIGGER
```

```
=> CREATE TRIGGER t_after_row
AFTER INSERT OR UPDATE OR DELETE ON t
FOR EACH ROW EXECUTE FUNCTION describe();
```

```
CREATE TRIGGER
```

Пробуем вставку:

```
=> INSERT INTO t VALUES (1,'aaa'), (2, 'bbb');
```

```
NOTICE:  t BEFORE INSERT STATEMENT:
NOTICE:  t BEFORE INSERT ROW: (1,aaa)
NOTICE:  t BEFORE INSERT ROW: (2,bbb)
NOTICE:  t AFTER INSERT ROW: (1,aaa)
NOTICE:  t AFTER INSERT ROW: (2,bbb)
NOTICE:  t AFTER INSERT STATEMENT:
INSERT 0 2
```

Обновление:

```
=> UPDATE t SET s = 'ccc' WHERE id = 1;
```

```
NOTICE:  t BEFORE UPDATE STATEMENT:
NOTICE:  t BEFORE UPDATE ROW: (1,aaa) -> (1,ccc)
NOTICE:  t AFTER UPDATE ROW: (1,aaa) -> (1,ccc)
NOTICE:  t AFTER UPDATE STATEMENT:
UPDATE 1
```

Триггеры на уровне оператора сработают, даже если команда не обработала ни одной строки:

```
=> UPDATE t SET s = 'ddd' WHERE id = 0;
```

```
NOTICE:  t BEFORE UPDATE STATEMENT:
NOTICE:  t AFTER UPDATE STATEMENT:
UPDATE 0
```

Тонкий момент: оператор INSERT с предложением ON CONFLICT приводит к тому, что срабатывают BEFORE-триггеры и на вставку, и на обновление:

```
=> INSERT INTO t VALUES (1,'ddd'), (3,'eee')
ON CONFLICT(id) DO UPDATE SET s = EXCLUDED.s;
```

```
NOTICE:  t BEFORE INSERT STATEMENT:
NOTICE:  t BEFORE UPDATE STATEMENT:
NOTICE:  t BEFORE INSERT ROW: (1,ddd)
NOTICE:  t BEFORE UPDATE ROW: (1,ccc) -> (1,ddd)
NOTICE:  t BEFORE INSERT ROW: (3,eee)
NOTICE:  t AFTER UPDATE ROW: (1,ccc) -> (1,ddd)
NOTICE:  t AFTER INSERT ROW: (3,eee)
NOTICE:  t AFTER UPDATE STATEMENT:
NOTICE:  t AFTER INSERT STATEMENT:
INSERT 0 2
```

И, наконец, удаление:

```
=> DELETE FROM t WHERE id = 2;
```

```
NOTICE:  t BEFORE DELETE STATEMENT:
NOTICE:  t BEFORE DELETE ROW: (2,bbb)
NOTICE:  t AFTER DELETE ROW: (2,bbb)
NOTICE:  t AFTER DELETE STATEMENT:
DELETE 1
```

Для появившегося в PostgreSQL 15 оператора MERGE специального триггера нет, работают триггеры на обновление, удаление и вставку:

```
=> MERGE INTO t
USING (VALUES (1, 'fff'), (3, 'ggg'), (4, 'hhh')) AS vals(id, s)
ON t.id = vals.id
WHEN MATCHED AND t.id = 1 THEN
  UPDATE SET s = vals.s
WHEN MATCHED THEN
  DELETE
WHEN NOT MATCHED THEN
  INSERT (id, s)
  VALUES (vals.id, vals.s);
```

```
NOTICE:  t BEFORE INSERT STATEMENT:
NOTICE:  t BEFORE UPDATE STATEMENT:
NOTICE:  t BEFORE DELETE STATEMENT:
NOTICE:  t BEFORE UPDATE ROW: (1,ddd) -> (1,fff)
NOTICE:  t BEFORE DELETE ROW: (3,eee)
NOTICE:  t BEFORE INSERT ROW: (4,hhh)
NOTICE:  t AFTER UPDATE ROW: (1,ddd) -> (1,fff)
NOTICE:  t AFTER DELETE ROW: (3,eee)
NOTICE:  t AFTER INSERT ROW: (4,hhh)
NOTICE:  t AFTER DELETE STATEMENT:
NOTICE:  t AFTER UPDATE STATEMENT:
NOTICE:  t AFTER INSERT STATEMENT:
MERGE 3
```

Переходные таблицы

Напишем триггерную функцию, показывающую содержимое переходных таблиц. Здесь мы используем имена old_table и new_table, которые будут объявлены при создании триггера.

Переходные таблицы «выглядят» настоящими, но не присутствуют в системном каталоге и располагаются в оперативной памяти (хотя и могут сбрасываться на диск при большом объеме).

```
=> CREATE OR REPLACE FUNCTION transition() RETURNS trigger
AS $$
DECLARE
    rec record;
BEGIN
    IF TG_OP = 'DELETE' OR TG_OP = 'UPDATE' THEN
        RAISE NOTICE 'Старое состояние:';
        FOR rec IN SELECT * FROM old_table LOOP
            RAISE NOTICE '%', rec;
        END LOOP;
    END IF;
    IF TG_OP = 'UPDATE' OR TG_OP = 'INSERT' THEN
        RAISE NOTICE 'Новое состояние:';
        FOR rec IN SELECT * FROM new_table LOOP
            RAISE NOTICE '%', rec;
        END LOOP;
    END IF;
    RETURN NULL;
END
$$ LANGUAGE plpgsql;
```

```
CREATE FUNCTION
```

Создадим новую таблицу:

```
=> CREATE TABLE trans(
    id integer PRIMARY KEY,
    n integer
);
```

```
CREATE TABLE
```

```
=> INSERT INTO trans VALUES (1,10), (2,20), (3,30);
```

```
INSERT 0 3
```

Чтобы при выполнении операции создавались переходные таблицы, необходимо указывать их имена при создании триггера:

```
=> CREATE TRIGGER t_after_upd_trans
AFTER UPDATE ON trans -- только одно событие на один триггер
REFERENCING
    OLD TABLE AS old_table
    NEW TABLE AS new_table -- можно и одну, не обязательно обе
FOR EACH STATEMENT
EXECUTE FUNCTION transition();
```

```
CREATE TRIGGER
```

Проверим:

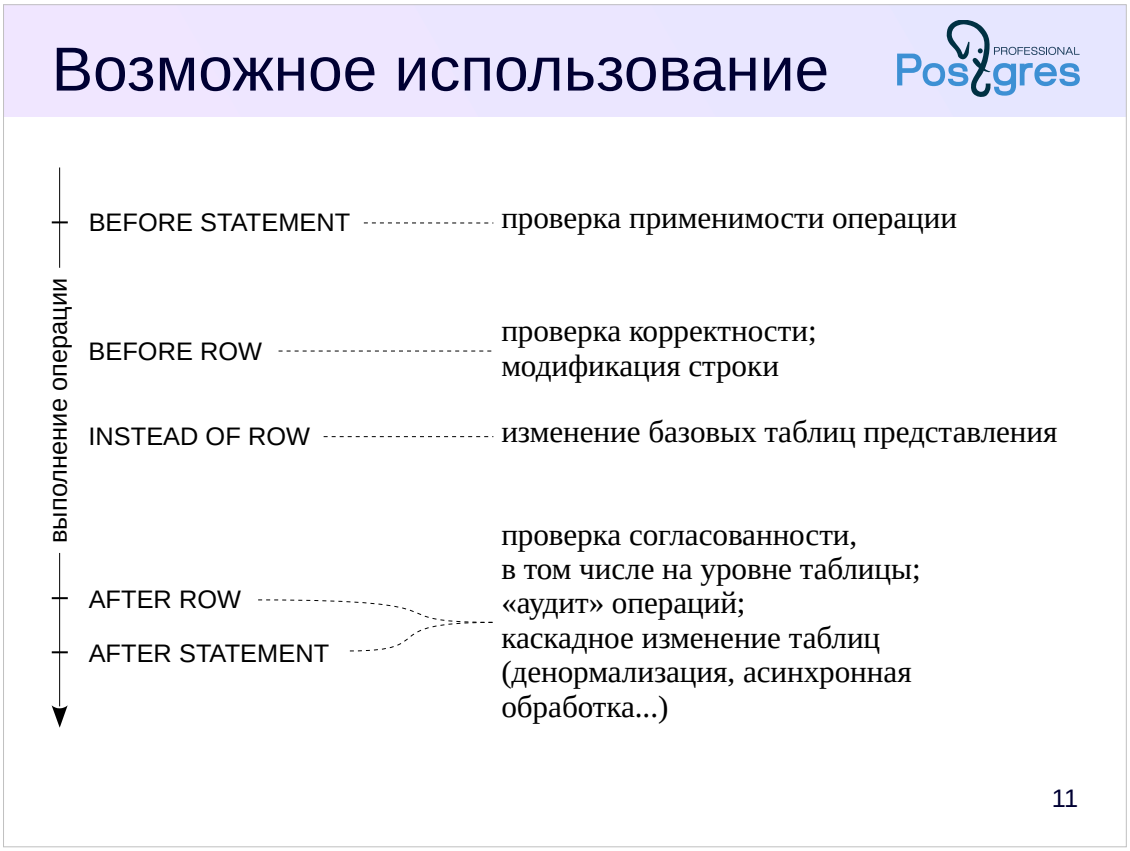
```
=> UPDATE trans SET n = n + 1 WHERE n <= 20;
```

```
NOTICE:  Старое состояние:
NOTICE:  (1,10)
NOTICE:  (2,20)
NOTICE:  Новое состояние:
NOTICE:  (1,11)
NOTICE:  (2,21)
UPDATE 2
```

Переходные таблицы содержат только те строки, которые были затронуты операцией.

Для операций вставки и удаления переходные таблицы работают точно так же, но доступна будет только одна из таблиц: NEW TABLE или OLD TABLE соответственно.

Поскольку триггеры AFTER ROW срабатывают после выполнения всей операции, переходные таблицы можно использовать и в них. Но обычно это не имеет смысла.



Каково практическое применение триггеров?

BEFORE-триггеры можно использовать для проверки корректности операции и при необходимости вызывать ошибку.

Триггеры BEFORE ROW можно применять для модификации строки (например, заполнить пустое поле нужным значением). Это бывает удобно, чтобы не повторять логику заполнения «технических» полей в каждой операции, а также позволяет вмешаться в работу приложения, код которого недоступен для изменения.

Триггеры INSTEAD OF ROW применяют для того, чтобы отобразить операции над представлением в операции над базовыми таблицами.

Триггеры AFTER ROW и AFTER STATEMENT полезны в случаях, когда нужно знать точное состояние после операции (BEFORE-триггеры могут влиять на результат, так что на этом этапе ясности еще нет):

- для проверки согласованности операции;
- для «аудита», то есть записи изменений в отдельное хранилище;
- для каскадного изменения других таблиц (например, обновлять денормализованные данные при изменении базовых таблиц, или записывать изменения в очередь для последующей обработки вне данной транзакции).

Если операции затрагивают сразу много строк, то триггер AFTER STATEMENT с переходными таблицами может оказаться более эффективным решением, чем триггер AFTER ROW, поскольку позволяет обрабатывать все изменения пакетно, а не построчно.

Сложности

Код вызывается неявно

сложно отследить логику выполнения

Правила видимости изменчивой триггерной функции

виден результат триггеров BEFORE ROW или INSTEAD OF ROW

Порядок вызова триггеров для одного события

триггеры отрабатывают в алфавитном порядке

Не предотвращается зацикливание

триггер может вызвать срабатывание других триггеров

Можно нарушить ограничения целостности

например, исключив из обработки строки, которые должны удалиться

Однако не стоит злоупотреблять триггерами. Триггеры срабатывают неявно, что сильно затрудняет понимание логики работы и крайне усложняет поддержку приложения. Попытки реализовать сложную логику на триггерах обычно заканчиваются плачевно.

В некоторых случаях вместо триггеров можно использовать вычисляемые поля (GENERATED ALWAYS AS … STORED). Такое решение — если оно подходит — будет заведомо проще и прозрачнее.

Есть ряд тонкостей, связанных с триггерами, которые мы сознательно не рассматриваем подробно:

- правила видимости изменчивых (volatile) функций в триггерах BEFORE ROW и INSTEAD OF ROW (не стоит обращаться к таблице, полагаясь на порядок, в котором сработают триггеры);
- порядок вызова нескольких триггеров, обрабатывающих одно и то же событие (не стоит усугублять и без того неявное срабатывание триггеров завязкой на последовательность обработки);
- возможность зацикливания в случае каскадного срабатывания других триггеров, которые, в свою очередь, могут приводить к новым срабатываниям данного триггера;
- возможность нарушить ограничения целостности (например, при исключении из обработки строки, которая удаляется условием ON DELETE CASCADE, может быть нарушена ссылочная целостность).

Если вы столкнулись с тем, что эти тонкости важны для вашего приложения — серьезно задумайтесь.

Примеры использования триггеров

Пример 1: сохранение истории изменения строк.

Пусть есть таблица, содержащая актуальные данные. Задача состоит в том, чтобы в отдельной таблице сохранять всю историю изменения строк основной таблицы.

Поддержку исторической таблицы можно было бы возложить на приложение, но тогда велика вероятность, что в каких-то случаях из-за ошибок история не будет сохраняться. Поэтому решим задачу с помощью триггера.

Основная таблица:

```
=> CREATE TABLE coins(
    face_value numeric PRIMARY KEY,
    name text
);
```

```
CREATE TABLE
```

Историческая таблица должна называться так же, как основная, но заканчиваться на «_history». Сначала создаем клон основной таблицы...

```
=> CREATE TABLE coins_history(LIKE coins);
```

```
CREATE TABLE
```

...и затем добавляем столбцы «действительно с» и «действительно по»:

```
=> ALTER TABLE coins_history
    ADD start_date timestamp,
    ADD end_date timestamp;
```

```
ALTER TABLE
```

Одна триггерная функция будет вставлять новую историческую строку с открытым интервалом действия:

```
=> CREATE OR REPLACE FUNCTION history_insert() RETURNS trigger
AS $$
BEGIN
    EXECUTE format(
        'INSERT INTO %I SELECT ($1).*, current_timestamp, NULL',
        TG_TABLE_NAME||'_history'
    ) USING NEW;

    RETURN NEW;
END
$$ LANGUAGE plpgsql;
```

```
CREATE FUNCTION
```

Другая функция будет закрывать интервал действия исторической строки:

```
=> CREATE OR REPLACE FUNCTION history_delete() RETURNS trigger
AS $$
BEGIN
    EXECUTE format(
        'UPDATE %I SET end_date = current_timestamp WHERE face_value = $1 AND end_date IS NULL',
        TG_TABLE_NAME||'_history'
    ) USING OLD.face_value;

    RETURN OLD;
END
$$ LANGUAGE plpgsql;
```

```
CREATE FUNCTION
```

Теперь создадим триггеры. Важные моменты:

- Обновление трактуется как удаление и последующая вставка; здесь важен порядок, в котором сработают триггеры (по алфавиту).
- Current_timestamp возвращает время начала транзакции, поэтому при обновлении start_date одной строки будет равен end_date другой.
- Использование AFTER-триггеров позволяет избежать проблем с INSERT ... ON CONFLICT и потенциальными конфликтами с другими триггерами, которые могут существовать на основной таблице.

```
=> CREATE TRIGGER coins_history_insert
AFTER INSERT OR UPDATE ON coins
FOR EACH ROW EXECUTE FUNCTION history_insert();
```

```
CREATE TRIGGER
```

```
=> CREATE TRIGGER coins_history_delete
AFTER UPDATE OR DELETE ON coins
FOR EACH ROW EXECUTE FUNCTION history_delete();
```

```
CREATE TRIGGER
```

Проверим работу триггеров.

```
=> INSERT INTO coins VALUES (0.25, 'Полушка'), (3, 'Алтын');
```

```
INSERT 0 2
```

```
=> UPDATE coins SET name = '3 копейки' WHERE face_value = 3;
```

```
UPDATE 1
```

```
=> INSERT INTO coins VALUES (5, '5 копеек');
```

```
INSERT 0 1
```

```
=> DELETE FROM coins WHERE face_value = 0.25;
```

```
DELETE 1
```

```
=> SELECT * FROM coins;
```

```
 face_value |   name
------------+-----------
          3 | 3 копейки
          5 | 5 копеек
(2 rows)
```

В исторической таблице хранится вся история изменений:

```
=> SELECT * FROM coins_history ORDER BY face_value, start_date;
```

```
 face_value |   name    |         start_date         |          end_date
------------+-----------+----------------------------+----------------------------
       0.25 | Полушка   | 2025-06-24 04:36:34.481538 | 2025-06-24 04:36:39.686557
          3 | Алтын     | 2025-06-24 04:36:34.481538 | 2025-06-24 04:36:36.575901
          3 | 3 копейки | 2025-06-24 04:36:36.575901 |
          5 | 5 копеек  | 2025-06-24 04:36:37.646026 |
(4 rows)
```

И теперь по ней можно восстановить состояние на любой момент времени (это немного напоминает работу механизма MVCC). Например, на самое начало:

```
=> \set d '2025-06-24 04:36:34.524876+03'
```

```
=> SELECT face_value, name
FROM coins_history
WHERE start_date <= :'d' AND (end_date IS NULL OR :'d' < end_date)
ORDER BY face_value;
```

```
 face_value |  name
------------+---------
       0.25 | Полушка
          3 | Алтын
(2 rows)
```

Примеры использования триггеров

Пример 2: обновляемое представление.

Пусть имеются две таблицы: аэропорты и рейсы:

```
=> CREATE TABLE airports(
    code char(3) PRIMARY KEY,
    name text NOT NULL
);
```

```
CREATE TABLE
```

```
=> INSERT INTO airports VALUES
    ('SVO', 'Москва. Шереметьево'),
    ('LED', 'Санкт-Петербург. Пулково'),
    ('TOF', 'Томск. Богашево');
```

```
INSERT 0 3
```

```
=> CREATE TABLE flights(
    id integer PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
    airport_from char(3) NOT NULL REFERENCES airports(code),
    airport_to   char(3) NOT NULL REFERENCES airports(code),
    UNIQUE (airport_from, airport_to)
);
```

```
CREATE TABLE
```

```
=> INSERT INTO flights(airport_from, airport_to) VALUES
    ('SVO','LED');
```

```
INSERT 0 1
```

Для удобства можно определить представление:

```
=> CREATE VIEW flights_v AS
SELECT id,
       (SELECT name
        FROM airports
        WHERE code = airport_from) airport_from,
       (SELECT name
        FROM airports
        WHERE code = airport_to) airport_to
FROM flights;
```

```
CREATE VIEW
```

```
=> SELECT * FROM flights_v;
```

```
 id |    airport_from     |        airport_to
----+---------------------+--------------------------
  1 | Москва. Шереметьево | Санкт-Петербург. Пулково
(1 row)
```

Но такое представление не допускает изменений. Например, не получится изменить пункт назначения таким образом:

```
=> UPDATE flights_v
SET airport_to = 'Томск. Богашево'
WHERE id = 1;
```

```
ERROR:  cannot update column "airport_to" of view "flights_v"
DETAIL:  View columns that are not columns of their base relation are not updatable.
```

Однако мы можем определить триггер. Триггерная функция может выглядеть, например, так (для краткости обрабатываем только аэропорт назначения, но не составит труда добавить и аэропорт вылета):

```
=> CREATE OR REPLACE FUNCTION flights_v_update() RETURNS trigger
AS $$
DECLARE
    code_to char(3);
BEGIN
    BEGIN
        SELECT code INTO STRICT code_to
        FROM airports
        WHERE name = NEW.airport_to;
    EXCEPTION
        WHEN no_data_found THEN
            RAISE EXCEPTION 'Аэропорт "%" отсутствует', NEW.airport_to;
    END;
    UPDATE flights
    SET airport_to = code_to
    WHERE id = OLD.id; -- изменение id игнорируем
    RETURN NEW;
END
$$ LANGUAGE plpgsql;
```

```
CREATE FUNCTION
```

И сам триггер:

```
=> CREATE TRIGGER flights_v_upd_trigger
INSTEAD OF UPDATE ON flights_v
FOR EACH ROW EXECUTE FUNCTION flights_v_update();
```

```
CREATE TRIGGER
```

Проверим:

```
=> UPDATE flights_v
SET airport_to = 'Томск. Богашево'
WHERE id = 1;
```

```
UPDATE 1
```

```
=> SELECT * FROM flights_v;
```

```
 id |    airport_from     |   airport_to
----+---------------------+-----------------
  1 | Москва. Шереметьево | Томск. Богашево
(1 row)
```

Попытка изменить аэропорт на отсутствующий в таблице:

```
=> UPDATE flights_v
SET airport_to = 'Южно-Сахалинск. Хомутово'
WHERE id = 1;
```

```
ERROR:  Аэропорт "Южно-Сахалинск. Хомутово" отсутствует
CONTEXT:  PL/pgSQL function flights_v_update() line 11 at RAISE
```

Событийные триггеры

Событийный триггер

похож на обычный «табличный» триггер, но другой объект

Триггерная функция

соглашение: функция не принимает параметры,
возвращает значение псевдотипа event_trigger

для получения контекста служат специальные функции

События

DDL_COMMAND_START	перед выполнением команды
DDL_COMMAND_END	после выполнения команды
TABLE_REWRITE	перед перезаписью таблицы
SQL_DROP	после удаления объектов

Событийные триггеры — по сути те же триггеры, но срабатывают они не на DML-, а на DDL-операции (CREATE, ALTER, DROP, COMMENT, GRANT, REVOKE) и не привязаны к определенному объекту базы данных.

Такие триггеры являются не инструментом разработки приложений, а скорее служат для решения задач администрирования. Поэтому здесь мы упоминаем их только для полноты картины и рассмотрим только простые примеры.

https://postgrespro.ru/docs/postgresql/16/event-trigger-definition

https://postgrespro.ru/docs/postgresql/16/event-trigger-matrix

https://postgrespro.ru/docs/postgresql/16/sql-createeventtrigger

https://postgrespro.ru/docs/postgresql/16/functions-event-triggers

Триггеры событий

Пример триггера для события ddl_command_end — завершение DDL-операции.

Создадим функцию, которая описывает контекст вызова:

```
=> CREATE OR REPLACE FUNCTION describe_ddl() RETURNS event_trigger
AS $$
DECLARE
    r record;
BEGIN
    -- Для события ddl_command_end контекст вызова в специальной функции
    FOR r IN SELECT * FROM pg_event_trigger_ddl_commands()
    LOOP
        RAISE NOTICE '%. тип: %, OID: %, имя: % ',
            r.command_tag, r.object_type, r.objid, r.object_identity;
    END LOOP;
    -- Функции триггера событий не нужно возвращать значение
END
$$ LANGUAGE plpgsql;
```

```
CREATE FUNCTION
```

Сам триггер:

```
=> CREATE EVENT TRIGGER after_ddl
ON ddl_command_end EXECUTE FUNCTION describe_ddl();
```

```
CREATE EVENT TRIGGER
```

Создаем новую таблицу:

```
=> CREATE TABLE t1(id integer PRIMARY KEY GENERATED ALWAYS AS IDENTITY);
```

```
NOTICE:  CREATE SEQUENCE. тип: sequence, OID: 16459, имя: public.t1_id_seq
NOTICE:  CREATE TABLE. тип: table, OID: 16460, имя: public.t1
NOTICE:  CREATE INDEX. тип: index, OID: 16463, имя: public.t1_pkey
NOTICE:  ALTER SEQUENCE. тип: sequence, OID: 16459, имя: public.t1_id_seq
CREATE TABLE
```

Создание таблицы может привести к выполнению нескольких команд DDL, поэтому функция pg_event_trigger_ddl_commands возвращает множество строк.

Триггер на событие table_rewrite срабатывает до перезаписи таблицы командами ALTER TABLE и ALTER TYPE. Функция ниже выведет информацию о перезаписываемой таблице и код, описывающий причину перезаписи:

```
=> CREATE FUNCTION describe_rewrite() RETURNS event_trigger
AS $$
BEGIN
    -- Для события table_rewrite специальные функции возвращают данные:
    RAISE NOTICE 'Будет перезаписана таблица %, код %',
                pg_event_trigger_table_rewrite_oid()::regclass,  -- перезаписываемая таблица
                pg_event_trigger_table_rewrite_reason();  -- код причины перезаписи
END
$$ LANGUAGE plpgsql;
```

```
NOTICE:  CREATE FUNCTION. тип: function, OID: 16465, имя: public.describe_rewrite()
CREATE FUNCTION
```

Теперь создаем триггер:

```
=> CREATE EVENT TRIGGER before_rewrite
ON table_rewrite EXECUTE FUNCTION describe_rewrite();
```

```
CREATE EVENT TRIGGER
```

Изменим таблицу — назначим другой тип столбцу id:

```
=> ALTER TABLE t1 ALTER COLUMN id type bigint;
```

```
NOTICE:  Будет перезаписана таблица t1, код 4
NOTICE:  ALTER SEQUENCE. тип: sequence, OID: 16459, имя: public.t1_id_seq
NOTICE:  ALTER TABLE. тип: table, OID: 16460, имя: public.t1
ALTER TABLE
```

Хотя перезапись таблицы может быть вызвана и другими командами, в частности CLUSTER и VACUUM FULL, событие table_rewrite для них не вызывается.

Итоги

Триггер — способ отреагировать на возникновение события

С помощью триггера можно отменить операцию, изменить ее результат или выполнить дополнительные действия

Триггер выполняется как часть транзакции; ошибка в триггере приводит к откату транзакции

Использование триггеров AFTER ROW и переходных таблиц удорожает обработку

Все хорошо в меру: сложную логику трудно отлаживать из-за неявного выполнения триггеров